

{EPITECH}

SENSAI

< WATASHI WA NIHONGO O SUKOSHI SHIKA
HANASEMASEN />





language: Python 3.10 or higher

authorized frameworks: No LLM framework (LangChain, LlamaIndex, Haystack, or equivalent). You may use standard libraries and utility packages (requests, sqlite3, fastapi, etc.) for non-LLM concerns.

Ollama API: All calls to the model must go through the Ollama HTTP API directly.

Introduction

Every AI assistant you have used is built on the same foundation: a **language model**, a **context window**, and a **loop**. What makes them powerful is not (only) the model itself, but the architecture built around it: memory systems, tool integrations, evaluation pipelines, safety mechanisms... NotebookLM lets you interrogate your own documents because someone built a RAG pipeline. Cursor corrects its own code because someone built a reflection loop. Perplexity cites live sources because someone built a web search integration.

In this project, your goal is to build the most capable AI assistant you can, starting from a minimal local chatbot and enriching it with features of your choosing. You select your features, you argue your technical tradeoffs, and you justify every significant decision as a response to real use cases: A medical assistant that detects PII before sending data to the model, a developer agent that executes and corrects its own code in a sandbox, or maybe the best japanese teacher in all of history...

Base Loop

Your project **must** implement the following:

Functional CLI chatbot connected to a local Ollama model, with:

- **Streaming responses:** (output rendered progressively, not as a single block)
- **In-session conversation history:** (the model receives prior turns as context)
- **Clean error handling:** (unavailable model, empty input, connection failure)
- **Model selectable** via CLI argument or configuration file, without modifying the code



A Base loop should be what you have at the end of the bootstrap. This alone won't net you any points on the project.

Additional features

Beyond the MVP, you are free to implement any features from the catalogue. Features are grouped by domain and rated on two axes:

Difficulty (1–3): implementation effort

Pedagogical value (1–3): depth of learning

The full feature catalogue with descriptions, real-world examples, and ratings is available in the companion document.

Deliverables

Your repository must include at minimum:

- All source code
- A README.md with setup instructions, feature list, and usage examples
- A requirements.txt or equivalent dependency specification
- Your user stories



User Stories

For each feature you implement, you must write at least two user stories demonstrating that your system addresses real use cases.

User stories follow this format:

As a type of user, I want to action, so that outcome.

Acceptance criteria:

- Observable behavior 1
- Observable behavior 2
- Observable behavior 3

User stories are not a formality. They are the contract between your technical choices and the problem you claim to solve. A feature without a credible use case is a feature without justification.

Keynote

Your final presentation is a **keynote**.

What is expected:

- Open with a **concrete scenario**: who is using your system, for what purpose, in what context. Choose a real use case for your project.
- Demonstrate your features live, in that scenario. Do not switch between disconnected demos, show a coherent system.
- For each significant feature or groupe of features, explain the technical choices you made and why. What did you consider? What did you rule out?
- Close with an honest assessment: what works well, what are the current limits, what would you do differently.



Prepare your demos. A live system that crashes or hangs during a keynote is a preparation failure, not a technical one. Have fallback screenshots or recordings ready for anything that depends on network or heavy computation.

As always, have fun !

v1

{EPITECH}